Chapter 1

EM for the transition law from noised sequences

This chapter assembles the paper's estimator from the pieces of the previous four. The data are M noised sequences, each observed once at its own noise level; the unknown is the transition kernel K_θ of the clean chain; the objective is the marginal likelihood; the algorithm is expectation–maximisation with the exact E-step of **??** on the grid of **??**. Everything the M-step needs is one $N_{\mathrm{g}} \times N_{\mathrm{g}}$ matrix, the expected transition statistic $\boldsymbol{\Xi}$, and Fisher's identity turns the same matrix into the exact gradient of the marginal likelihood. The chapter derives Fisher's identity in full, defines $\boldsymbol{\Xi}$ and its properties, derives the Gaussian M-step (the belief-weighted Yule–Walker equations), describes the mixture-innovation kernel and its conditional-maximisation sweeps, explains the checkpoint ladder, the validation selection and the `at_cap` flag, the clean-data limit, and how E06, E06c and E02a use all of it. It ends with the information-theoretic reason for the convergence asymmetry the paper reports.

1.1 The marginal likelihood of noised sequences

The paper's Section 4 drops the assumption that K is known and keeps everything else: $a_1 \sim \mathcal{N}(0,1)$, $a_i = \rho a_{i-1} + \varepsilon_i$ with a parameterised transition kernel $K_\theta(a' \mid a)$, and the channel $\mathbf{x} = e^{-t}\mathbf{a} + \sqrt{\Delta_t}\mathbf{z}$. The observations are M sequences $\mathbf{x}^\mu$, the μ-th at level t_μ. Under the **one-view protocol** each clean sequence is noised once, at one level, and never seen clean; in the headline experiments the sequences are split evenly across the twelve levels of the schedule, and in the learning experiment E06 all of them sit at one level. The **marginal log-likelihood** is

$$\mathcal{L}(\theta) = \sum_{\mu=1}^{M} \log P_t[t_\mu, \theta](\mathbf{x}^\mu), \qquad P_t[t,\theta](\mathbf{x}) = \int \mu(a_1) \prod_{i=2}^{L} K_\theta(a_i \mid a_{i-1}) \prod_{i=1}^{L} \ell_t(x_i \mid a_i)\, \mathrm{d}\mathbf{a}, \quad (1.1)$$

where the unary factors ℓ_t depend on t_μ but not on θ, and only the transition factors carry θ. Each term of Equation (1.1) is the evidence of **??**, returned by the forward sweep on the grid. Two structural facts drive everything below: the parameter enters only through the $L-1$ transition factors of each sequence, and the noise level enters only through the unary factors, so sequences at different levels contribute additively and identically to every statistic (`e_step_multi` in `src/markov_score/em.py` simply adds them).

1.2 Fisher's identity

1.2.1 Derivation

Let $p_\theta(x, z)$ be a joint density of data and latent variables, with $p_\theta(x) = \int p_\theta(x, z)\,\mathrm{d}z$, and assume differentiation under the integral sign is permitted (the integrand and its θ-derivative are dominated by an integrable function, which holds for the kernels used here on the truncated grid, where the integral is a finite sum). Then

$$\begin{aligned}\nabla_\theta \log p_\theta(x) &= \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)} = \frac{1}{p_\theta(x)} \int \nabla_\theta p_\theta(x, z)\,\mathrm{d}z \\ &= \int \frac{p_\theta(x, z)}{p_\theta(x)}\, \nabla_\theta \log p_\theta(x, z)\,\mathrm{d}z = \mathbb{E}_{p_\theta(z \mid x)}\big[\nabla_\theta \log p_\theta(x, z)\big],\end{aligned} \tag{1.2}$$

using $\nabla p = p \nabla \log p$ in the third step and $p_\theta(x, z)/p_\theta(x) = p_\theta(z \mid x)$ in the last. This is **Fisher's identity**: the gradient of the marginal log-likelihood equals the posterior expectation of the gradient of the complete-data log-likelihood. It was met in **??** as the statement that EM's fixed points are stationary points; here it is a computational device.

Apply it to one sequence of Equation (1.1). The complete-data log-likelihood is $\log \mu(a_1) + \sum_{i \geq 2} \log K_\theta(a_i \mid a_{i-1}) + \sum_i \log \ell_t(x_i \mid a_i)$, and only the middle sum depends on θ, so

$$\begin{aligned}\nabla_\theta \log P_t[t, \theta](\mathbf{x}) &= \sum_{i=2}^{L} \mathbb{E}\big[\nabla_\theta \log K_\theta(a_i \mid a_{i-1}) \,\big|\, \mathbf{x}\big] \\ &= \sum_{i=2}^{L} \int p_t(a_{i-1}, a_i \mid \mathbf{x})\, \nabla_\theta \log K_\theta(a_i \mid a_{i-1})\,\mathrm{d}a_{i-1}\,\mathrm{d}a_i.\end{aligned} \tag{1.3}$$

Each term needs only the *pairwise* posterior marginal on one edge, which the two sweeps return exactly (equation *pair* of **??**). No derivative of the recursion is taken: the sweeps supply an expectation, and the gradient is an integral against it. This is the sense of the paper's "no differentiation through the recursion is needed".

1.2.2 On the grid: the expected transition statistic

Replace the integrals by the discretised model of **??**. The pairwise posterior on edge $(i-1, i)$ of sequence μ becomes a matrix of probabilities $P(a_i = u_k, a_{i-1} = u_l \mid \mathbf{x}^\mu)$ over grid pairs, and $\nabla_\theta \log K_\theta$ becomes the array $\nabla_\theta \log K_\theta(u_k \mid u_l)$. Summing over edges and sequences,

$$\begin{aligned}\nabla_\theta \mathcal{L}(\theta) &= \sum_{k,l} \Xi_{kl}(\theta)\, \nabla_\theta \log K_\theta(u_k \mid u_l) = \big\langle \Xi(\theta), \nabla_\theta \log K_\theta \big\rangle, \\ \Xi_{kl} &= \sum_{\mu} \sum_{i \geq 2} P(a_i = u_k, a_{i-1} = u_l \mid \mathbf{x}^\mu),\end{aligned} \tag{1.4}$$

which is the paper's equation *fisher*. The matrix Ξ is the **expected transition statistic**: entry (k, l) is the expected posterior mass on the transition $u_l \to u_k$ (column the parent, row the child, the same orientation as the transition matrix), summed over every edge of every sequence. It is the continuous-state analogue of the expected transition counts of Baum–Welch. Its computation from the sweeps is a rank-one accumulation: with $f_i = w \circ \overrightarrow{m}_i \circ \ell_i$ and $g_i = w \circ \overleftarrow{m}_i \circ \ell_i$ (messages with the local factor and the quadrature weight folded in), the unnormalised pairwise belief on edge $(i, i+1)$ is $f_i(l)\, K[k, l]\, g_{i+1}(k)$ with normaliser $Z_i = g_{i+1}^\top K f_i$, so

$$\Xi = C \circ K, \qquad C = \sum_{\mu} \sum_{i} \frac{g_{i+1}^{\mu}\, (f_i^{\mu})^\top}{Z_i^{\mu}}, \tag{1.5}$$

with $\circ$ the elementwise product. The code (`_e_step_chunk` in `src/markov_score/em.py`) forms C as one matrix product of the stacked g and f vectors over all edges of a chunk, then multiplies by K.

Three properties of $\boldsymbol{\Xi}$ are used constantly.

1. *Mass.* $\sum_{kl} \boldsymbol{\Xi}_{kl} = M(L-1)$, the number of transitions, because each pairwise posterior sums to one. This is checked exactly in `tests/test_em_bp.py` and is what lets the clean-data histogram of Section 1.6 be a drop-in replacement.

2. *Additivity.* Sequences at different levels contribute additively; one $\boldsymbol{\Xi}$ summarises a dataset spread over the whole schedule.

3. *What it is not.* $\boldsymbol{\Xi}$ is a complete summary for the current M-step (the maximisation never revisits the data), but it is not a sufficient statistic of the observations: it is computed at the current θ and depends on it, on the grid, on the fixed initial law and on the homogeneity of the transition. The code's docstring insists on this distinction, and so should the reader.

1.2.3 The same matrix is the whole E-step

With $\boldsymbol{\Xi}$ in hand, the Q-function of **??** for the discretised model is

$$\mathcal{Q}(\theta \mid \theta') = \langle \boldsymbol{\Xi}(\theta'), \log K_\theta \rangle + (\text{site-1 term}) + \text{const}, \tag{1.6}$$

a finite bilinear form; the site-1 term involves the initial law, which the paper holds fixed at $\mathcal{N}(0,1)$, and the constant collects the likelihood terms and the posterior entropy, none of which depend on θ. Its gradient at $\theta = \theta'$ is Equation (1.4): the Q-function's gradient and the marginal likelihood's gradient coincide where the E-step was taken, which is Fisher's identity once more. The E-step is therefore "one forward–backward pass" plus the accumulation Equation (1.5), at cost $O(MLN_{\mathrm{g}}^2)$ independent of the number of parameters, and the M-step touches only $\boldsymbol{\Xi}$, at cost $O(PN_{\mathrm{g}}^2)$ for P parameters.

1.2.4 Verification

The paper states that the identity is verified against finite differences to 10^{-9}. Three tests do the work, and it matters which checks what. `test_fisher_identity_matches_finite_differences` in `tests/test_em_bp.py` compares $\langle \boldsymbol{\Xi}, \nabla \log K_\theta \rangle$ with a central difference of the grid evidence for the Gaussian kernel: a real check, but not an independent one, since the evidence and $\boldsymbol{\Xi}$ come from the same recursion and the same discretised kernel, so a shared error (a transposed index, a misplaced factor) could satisfy both sides. `tests/test_fisher_bruteforce.py` removes that coupling by enumerating every latent path of a five-state, four-site chain from the model definition, with no recursion, and checking the evidence, $\boldsymbol{\Xi}$ *including its orientation* (the matrix must match and must not match its transpose, on a grid asymmetric enough that the two differ), and the gradient against a finite difference of the *enumerated* evidence. The Laplace kernel is included there deliberately: its ρ-derivative carries $\operatorname{sign}(a' - \rho a)$, a discontinuity along the diagonal, and trapezoidal quadrature of a discontinuous integrand is first order (**??**); the test in `tests/test_em_bp.py` named for the Laplace scale gradient pins the fact that the smooth b-derivative matches a finite difference to high accuracy while the ρ-derivative is off by a grid-dependent percentage. That is a discretisation artefact of differentiating a non-smooth density on a mesh, not a failure of the identity; the Laplace M-step avoids it by never using the gradient (it is a weighted median). Finally, `tests/test_gradient_trace_alignment.py` guards a different invariant of the gradient-ascent route: the returned parameter is the one at which the last recorded likelihood was evaluated, and the monotonicity record covers the final update.

1.3 The M-step for the Gaussian kernel

Take $K_\theta(a' \mid a) = \mathcal{N}(\rho a, q)$, $\theta = (\rho, q)$. The edge part of Equation (1.6) is

$$\mathcal{Q}(\rho, q) = \sum_{k,l} \Xi_{kl} \Big[-\frac{(u_k - \rho u_l)^2}{2q} - \frac{1}{2}\log(2\pi q) \Big] = -\frac{S_{11} - 2\rho S_{01} + \rho^2 S_{00}}{2q} - \frac{W}{2}\log(2\pi q), \quad (1.7)$$

where $W = \sum_{kl} \Xi_{kl}$ and the three belief-weighted moments are

$$S_{00} = \sum_{kl} \Xi_{kl}\, u_l^2, \qquad S_{01} = \sum_{kl} \Xi_{kl}\, u_k u_l, \qquad S_{11} = \sum_{kl} \Xi_{kl}\, u_k^2,$$

the second moment of the parent, the cross moment, and the second moment of the child. The subscripts index the two variables of a transition (0 for the parent u_l, 1 for the child u_k) and say which pair is multiplied; they are not powers. The paper's Section 4 writes the same three quantities as $S_{ab} = \sum_{kl} \Xi_{kl} u_k^a u_l^b$, which read literally as powers would make $S_{00} = W$ and $S_{01} = \sum \Xi_{kl} u_l$ and would not give the formulas that follow; the index reading above is the one the formulas and the code use. Setting the derivatives to zero,

$$\begin{aligned} \frac{\partial \mathcal{Q}}{\partial \rho} &= \frac{S_{01} - \rho S_{00}}{q} = 0 && \Rightarrow \quad \rho = \frac{S_{01}}{S_{00}}, \\ \frac{\partial \mathcal{Q}}{\partial q} &= \frac{S_{11} - 2\rho S_{01} + \rho^2 S_{00}}{2q^2} - \frac{W}{2q} = 0 && \Rightarrow \quad q = \frac{S_{11} - 2\rho S_{01} + \rho^2 S_{00}}{W} = \frac{S_{11} - \rho S_{01}}{W}, \end{aligned} \quad (1.8)$$

the last equality because $\rho^2 S_{00} = \rho S_{01}$ at the optimum. These are the **belief-weighted Yule–Walker equations**: the least-squares regression of the child on the parent through the origin, with the pairwise posterior mass as weights, and the residual variance as the innovation variance. The Hessian of Equation (1.7) is negative definite at this point (it is a weighted least-squares problem in ρ and a concave profile in q), so this is the maximiser and the M-step is exact. In the clean-data limit, where Ξ is a histogram of observed transitions, Equation (1.8) reduces to the ordinary AR(1) least-squares estimator, which the clean-statistics test of `tests/test_em_bp.py` confirms. The Gaussian kernel's `m_step` in `src/markov_score/kernels.py` does not clamp $|\rho| < 1$: the update is the exact maximiser, a finite chain with $|\rho| \geq 1$ is a valid joint law, and what fails there is only the stationary interpretation $q = 1 - \rho^2$, which the code flags with a warning rather than silently biasing the step.

1.4 The mixture-innovation kernel

1.4.1 Definition and moments

The paper's headline estimator keeps the linear autoregression and frees the innovation law:

$$K_\theta(a' \mid a) = \sum_{c=1}^{C} \pi_c \mathcal{N}\Big(a' - \rho a - \nu_c, s_c^2\Big), \qquad \theta = (\rho, \pi_1, \ldots, \pi_C, \nu_1, \ldots, \nu_C, s_1^2, \ldots, s_C^2), \quad (1.9)$$

a C-component Gaussian mixture as the density of the innovation $\varepsilon = a' - \rho a$. Heavy tails (Laplace, Student-t), bimodality and skew are all approximable; the estimator never sees a Laplace density and must reproduce one from noised data. The component means are unconstrained: recentering them after each update to enforce $\sum_c \pi_c \nu_c = 0$ is not a maximiser of $\mathcal{Q}$ and was found to break monotone ascent, so the fitted innovation mean is reported as a diagnostic instead (on zero-mean data it lands near zero by itself). The fitted innovation's moments are read from the components in closed form,

$$\begin{aligned} m_1 &= \sum_c \pi_c \nu_c, \qquad q_{\text{fit}} = \sum_c \pi_c (s_c^2 + (\nu_c - m_1)^2), \\ m_4 &= \sum_c \pi_c (3 s_c^4 + 6 s_c^2 (\nu_c - m_1)^2 + (\nu_c - m_1)^4), \qquad \hat{\kappa}_4^{\text{ex}} = \frac{m_4}{q_{\text{fit}}^2} - 3, \end{aligned} \quad (1.10)$$

using the Gaussian moments $\mathbb{E}(\nu + sZ - m_1)^4$ expanded in powers of sZ (the odd terms vanish). The last quantity is the fitted excess kurtosis, the scalar "non-Gaussianity knob" the paper compares with the truth (+3 for Laplace); the formula is `innovation_moments` in `src/markov_score/kernels.py` and is tested against draws from the same mixture.

1.4.2 The complete-data problem has a second latent variable

For the Gaussian kernel the M-step was a closed-form maximisation because $\log K_\theta$ was a quadratic in θ. For Equation (1.9) it is the logarithm of a sum, and the maximiser of $\langle \Xi, \log K_\theta \rangle$ has no closed form. The remedy is the classical one: the mixture has its own latent variable, the component label, and given Ξ the M-step is itself an EM problem over that label. Define the **responsibilities** on the grid,

$$r_c(k,l) = \frac{\pi_c \, \mathcal{N}\big(u_k - \rho u_l - \nu_c, s_c^2\big)}{\sum_{c'} \pi_{c'} \, \mathcal{N}\big(u_k - \rho u_l - \nu_{c'}, s_{c'}^2\big)}, \tag{1.11}$$

the posterior probability of component c given the transition $u_l \to u_k$. A sweep of **conditional maximisations** (**??**) then alternates two blocks, each a closed-form maximiser of the inner Q-function with the other block fixed, with $e_{kl} = u_k - \rho u_l$ the residual:

$$\text{mixture block:} \qquad \pi_c = \frac{\langle \Xi, r_c \rangle}{W}, \quad \nu_c = \frac{\langle \Xi, r_c \circ e \rangle}{\langle \Xi, r_c \rangle}, \quad s_c^2 = \frac{\langle \Xi, r_c \circ (e - \nu_c)^2 \rangle}{\langle \Xi, r_c \rangle}; \tag{1.12}$$

$$\text{autoregression block:} \qquad \rho = \frac{\sum_{c,k,l} \Xi_{kl} \, r_c(k,l) \, u_l \, (u_k - \nu_c)/s_c^2}{\sum_{c,k,l} \Xi_{kl} \, r_c(k,l) \, u_l^2/s_c^2}, \tag{1.13}$$

the first being the usual Gaussian-mixture M-step with the weights $\Xi_{kl} r_c(k,l)$ and the second a weighted least-squares fit of the child on the parent with component-specific offsets ν_c and precisions $1/s_c^2$ (differentiate $\sum \Xi \, r_c[-(u_k - \rho u_l - \nu_c)^2/2s_c^2]$ in ρ). The responsibilities are recomputed between the blocks. Each block increases the inner Q-function, which by Jensen's inequality (the standard mixture-EM argument applied with Ξ as the "data weights") increases $\langle \Xi, \log K_\theta \rangle$; so the sweep ascends $\mathcal{Q}$, and by the monotonicity theorem of **??** the outer EM's marginal likelihood cannot decrease. This is the sense of the paper's "a generalised EM whose ascent is monotone" and of its citation of Meng and Rubin.

The implementation, the mixture kernel's `m_step` in `src/markov_score/kernels.py`, sweeps until the per-transition gain in $\langle \Xi, \log K_\theta \rangle / W$ falls below 10^{-8} or sixteen sweeps are spent, and records which happened on the returned kernel; a negative gain beyond round-off raises an error, since a conditional maximiser cannot decrease its objective. The docstring records why sixteen: with four sweeps the innovation *shape* was materially under-converged on short outer runs while ρ barely moved, the same asymmetry as the outer loop's (Section 1.8), and sixteen removes the interaction at a quarter of the cost of sixty-four. Because Ξ is fixed during the sweeps, each costs $O(CN_g^2)$ on a matrix that never grows with the dataset, which is why the M-step's cost per iteration is independent of M and why, at the operating grid, it is the M-step rather than the E-step that dominates a fit below a few thousand sequences.

1.4.3 Resolution

A mixture component can be narrower than a grid cell: the variance floor in the code is 10^{-6}, a standard deviation of 10^{-3}, against a spacing of 0.04. A spike narrower than the mesh can raise the quadrature likelihood without describing any feature of the innovation law. The diagnostic `scale_diagnostics` reports the narrowest fitted standard deviation in units of h and a flag `resolved` equal to $s_{\min} \geq 2h$; every reported cell carries it, the paper's Appendix C says such cells are flagged rather than dropped, and its capacity table marks them.

1.5 The checkpoint ladder and validation selection

1.5.1 The ladder

The protocol (`experiments/protocol.py`) fixes an iteration ladder

$$\{10, 20, 40, 60, 80, 120, 160, 220, 300, 400, 600, 800, 1200\}.$$

`fit_em` runs EM to the top rung and snapshots the kernel at each rung. The snapshot semantics are precise: at iteration k the kernel about to be scored is saved *before* its E-step, so the saved kernel at rung k is the one whose likelihood is the k-th trace entry; the final iterate is always available under its own index, so a caller asking for a rung past an early convergence break still receives the best kernel rather than an error. The likelihood tolerance (10^{-9} nats per transition, signed) can end a run before the top; at the paper's sizes it rarely does, and the run's `stop_reason` records which happened.

1.5.2 Selection and the `at_cap` flag

Which rung is reported is chosen on a **validation bundle**: sequences drawn from a random stream disjoint from the training stream, noised, and scored by the relative score error on the bulk sites (the four sites at each end trimmed) against the reference posterior means computed with the *true* kernel. The rung with the smallest validation error is selected and then scored, once, on a separate test bundle. The selected rung is compared with the top of the ladder, and the boolean `at_cap` records whether they coincide (`best == max(saved)` in `experiments/e02_capacity.py` and `experiments/e06_learning_through_noise.py`). A cell with `at_cap` true is one whose validation curve was still falling at the last rung, so its error is an upper bound on what a longer ladder would give; the paper reports the fraction of such cells (as the macro that its Section 4 calls the at-cap rate) and refuses to read a difference between two arms as final when either is at cap. E20 shows that the oracle in this rule is dispensable: selecting the rung by the held-out one-view log-evidence per transition, which needs neither the clean chain nor the true kernel, picks the same rung in most seeds and the same test error within noise.

1.6 The clean-data limit

As $t \to 0$ the unary factors sharpen to delta functions at the observations, the posterior collapses onto the clean chain, the pairwise beliefs become products of deltas, and Ξ degenerates to a histogram of the observed transitions. The code makes this limit an explicit input: `clean_statistics` in `src/markov_score/em.py` places each observed pair (a_{i-1}, a_i) on the grid by bilinear interpolation (a value a between u_k and u_{k+1} is split into weights $1-\theta$ and θ with $\theta = (a - u_k)/h$, which preserves the first moment exactly), and accumulates unit mass per transition. No quadrature weights enter: the clean Ξ is a counting measure, whereas the BP Ξ is belief mass that has already absorbed the weights; both sum to the number of transitions, which is the only property the M-steps rely on. `fit_clean` then applies the same M-step code, so the clean-data maximum-likelihood fit and the noisy-data EM differ only in how Ξ is built. The test `test_small_noise_em_approaches_clean_mle` pins the continuity between the two. E06 includes a clean arm as the "optimiser's own floor"; for one code path it is implemented as the channel at a tiny level, which the paper's Figure 3 shows as the dotted reference.

1.7 How the experiments use this

E02a (capacity). Laplace chain at the frozen configuration; for each seed and size M the same hashed bundles; mixture kernels with $C \in \{1, 2, 4, 8, 16\}$ from the same initialisation stream (ρ

initialised at 0.3, components spread over the innovation range), one view per sequence split over the twelve levels, the ladder above, the rung selected on validation identically for every capacity, then the test error in every region, the fitted ρ, innovation variance and excess kurtosis, the resolution flag and `at_cap`. The $C = 1$ arm must land on the LMMSE floor by the corollaries of **??**; the paper's capacity table (Section 5) reports that it does, and that the mixture goes far below it. The earlier reading that "capacity does not help" came from a sweep in which only the single-Gaussian arm stopped at its optimum (the tolerance fired for it) while the richer arms ran to a fixed cap past theirs: the stopping rule, not the model, produced the verdict (the paper's Appendix F).

E06 (learning through noise). The same estimator with $C = 8$ and $M = 512$, but with every training sequence noised at *one* level t, over a ladder of levels and sixteen replicates, the validation bundle at the same level, the test bundle on the twelve-level schedule. Recorded per cell: the selected rung, `at_cap`, the stop reason, $\hat{\rho}$, the fitted innovation variance and excess kurtosis, the Hellinger distance of the fitted kernel to the truth, the resolution flag and the schedule-pooled test error. The paper's Figure 3 and Section 4 read these against the Cramér–Rao reference $1/\sqrt{MI_t}$ level by level: unimodal and at the reference at low levels; a split into a shrunk cluster and Gaussian-collapsed fits at an intermediate level; collapse where the reference crosses the coordinate's range; $\hat{\rho}$ recovered until past the crossover. At the two highest levels the selected rung is the top of the ladder in most cells, which is the budget question.

E06c. The control that answers it: the same cells with the ladder extended sixfold. Its result is what decides whether the excess of $\hat{\rho}$'s error over the reference at those levels is information or optimisation; the paper leaves the question open pending it, and Section 1.8 says why the budget binds exactly there.

1.8 The convergence asymmetry

The paper reports, as a fact about this model, that the correlation coordinate settles in tens of EM updates and the innovation shape in hundreds. Two mechanisms produce it, and both can be read off the information at each Hermite degree.

The outer loop. By the rate formula of **??**, EM contracts the error along a coordinate by the factor $1 - I_{\text{obs}}/I_{\text{com}}$ per iteration, where I_{obs} is the information the noised data carry about the coordinate and I_{com} the information the clean chains would carry. The complete information is of the same order for both coordinates: clean transitions determine ρ and the innovation shape at the same rate in M. The observed information differs by the Hermite degree at which each coordinate first appears: by the paper's decay theorem, per sequence it is $\frac{1}{2}\|\partial_\rho \mathbf{\Sigma}_0\|_F^2 e^{-4t}$ for the correlation (degree two) and $\frac{1}{24}\|\partial_\theta \boldsymbol{\kappa}_4\|_F^2 e^{-8t}$ for a shape coordinate (degree four), up to corrections. So the missing fraction is closer to one for the shape at every level, by roughly a factor e^{-4t} in the retained fraction (0.37 at $t = 0.25$, 0.14 at $t = 0.5$, 0.018 at $t = 1$), and the number of iterations the shape needs grows correspondingly. In the clean limit both informations are complete and the asymmetry disappears, which the clean arm of E06 shows.

The inner loop. For the mixture kernel there is a second latent variable, the component label, and the shape parameters are learned through it: the M-step's inner sweeps are themselves an EM whose rate is governed by how well the responsibilities Equation (1.11) separate the components. A nearly Gaussian innovation gives nearly equal responsibilities, a missing fraction near one, and slow inner convergence, which the code's sixteen-sweep budget was introduced to absorb; ρ is not learned through the label at all (its block is a single least-squares solve) and does not suffer this. The two mechanisms compound, and both act on the same coordinate.

The consequence for the one-view protocol is the paper's arithmetic: of the twelve levels, spread over $[0.05, 3]$, only the lowest few carry appreciable information about the shape, since

e^{-8t} has fallen by a factor of several hundred by $t = 0.7$, so for the shape the twelve-level schedule is effectively a four-level one, and the estimator's shape is learned from a quarter of the data at a rate set by the smallest of its information ratios. That is why the ladder is long, why the shape is what the inner budget affects, and why the validation-selected rung sits at the cap precisely at the levels where the information is smallest.

1.9 A worked example: the Gaussian M-step from a small Ξ

Return to the three-site, three-state chain of **??**, grid $\{-1, 0, 1\}$. Its two pairwise posteriors summed over the two edges give, for that one sequence,

$$\Xi = \begin{pmatrix} 0.0802 & 0.1585 & 0.0665 \\ 0.0661 & 0.7356 & 0.2385 \\ 0.0251 & 0.2638 & 0.3656 \end{pmatrix}, \qquad W = \sum_{kl} \Xi_{kl} = 2,$$

rows indexed by the child u_k, columns by the parent u_l (the first edge matrix was written out in that chapter; the second is obtained the same way). The belief-weighted moments are

$$\begin{aligned} S_{00} &= \sum_{kl} \Xi_{kl} u_l^2 = (0.0802 + 0.0661 + 0.0251) + (0.0665 + 0.2385 + 0.3656) = 0.8421, \\ S_{11} &= \sum_{kl} \Xi_{kl} u_k^2 = (0.0802 + 0.1585 + 0.0665) + (0.0251 + 0.2638 + 0.3656) = 0.9598, \\ S_{01} &= \sum_{kl} \Xi_{kl} u_k u_l = 0.0802 - 0.0665 - 0.0251 + 0.3656 = 0.3542, \end{aligned}$$

where S_{01} collects the four corner entries with the sign of $u_k u_l$. Then Equation (1.8) gives $\rho = 0.3542/0.8421 = 0.4206$ and $q = (0.9598 - 0.4206 \cdot 0.3542)/2 = 0.4054$. Evaluating $\mathcal{Q}(\rho, q)$ of Equation (1.7) at this point gives -1.9350, and at the perturbed points (0.4706, 0.4054), (0.4206, 0.4459) and (0.3706, 0.3649) it gives -1.9376, -1.9394 and -1.9436: the closed form is the maximiser, as the test of the Gaussian M-step in `tests/test_em_bp.py` checks by random perturbation. A two-component mixture started at $\pi = (\frac{1}{2}, \frac{1}{2})$, $\nu = (-0.3, 0.3)$, $s^2 = (0.5, 0.5)$, $\rho = 0.2$ has $\langle \Xi, \log K_\theta \rangle / W = -1.0155$; one mixture block Equation (1.12) moves it to -0.9700 and the following autoregression block Equation (1.13) to -0.9592, with $\rho = 0.328$ after one sweep: each block increases the objective, as the conditional-maximisation argument requires. (The example has a single sequence and a three-point grid, so the numbers illustrate the algebra, not the statistics.)

Where it is used. The paper, Section 4 (the marginal likelihood, Fisher's identity and equation *fisher*, the Yule–Walker M-step, the mixture kernel and its conditional-maximisation sweeps, the one-view protocol and validation selection, the convergence asymmetry and the twelve-level arithmetic), Appendix C (the ladder and its selection rule, the resolution flag), Appendix E (E19's moment estimators as the competitor, E20's oracle-free selection), Appendix F (the stopping-rule artefact of the earlier capacity reading). Code: the E-step, its multi-level sum, the clean-data statistics, the two fitting loops and the Q-function helpers are all in `src/markov_score/em.py`; the three kernels, with the mixture's `innovation_moments` and `scale_diagnostics`, are in `src/markov_score/kernels.py`; the ladder `em_checkpoints` is in `experiments/protocol.py`; the drivers are `experiments/e02_capacity.py`, `experiments/e06_learning_through_noise.py` and `experiments/e20_evidence_selection.py`. Tests: `tests/test_em_bp.py`, `tests/test_fisher_bruteforce.py` and `tests/test_gradient_trace_alignment.py`.

References. A. P. Dempster, N. M. Laird and D. B. Rubin (1977), "Maximum likelihood from incomplete data via the EM algorithm", *Journal of the Royal Statistical Society B* 39(1). X.-L. Meng and D. B. Rubin (1993), "Maximum likelihood estimation via the ECM algorithm: a

general framework", *Biometrika* 80(2). O. Cappé, E. Moulines and T. Rydén (2005), *Inference in Hidden Markov Models*, Springer, Chapters 10 and 11 (Fisher's identity, EM for HMMs). R. H. Shumway and D. S. Stoffer (1982), "An approach to time series smoothing and forecasting using the EM algorithm", *Journal of Time Series Analysis* 3(4). R. Douc, E. Moulines, J. Olsson and R. van Handel (2011), "Consistency of the maximum likelihood estimator for general hidden Markov models", *Annals of Statistics* 39(1).